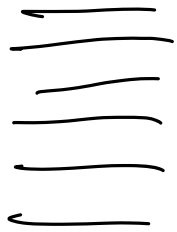
Scrittura sul file

- scrivi codice
- compilare
- eseguire
- guardare lo schermo

```
$ ./app.exe
```



```
$ ./app.exe > output.txt
```

contiene tutto il testo destinato allo schermo

grave.c

```c
#include <stdio.h>
#define GRAV 9.81

int main() {
    double h, dt, t, y;
    printf("altezza h (m): ");
    scanf("%lf", &h);
    printf("passo dt (s): ");
    scanf("%lf", &dt);
    t = 0.;
    y = h;
    printf("   t\t   y\n");
    while (y > 0.) {
        y = h - 0.5 * GRAV * t * t;
        printf("%6.2f\t%6.2f\n", t, y);
        t += dt;
    }
}
```

Terminale: grave.c

```
$ gcc -Wall -Wextra -o app.exe grave.c -lm
$ ./app.exe
altezza h (m): 20
passo dt (s): 0.1
   t       y
  0.00   20.00
  0.10   19.95
  0.20   19.80
  0.30   19.56
   ...
  2.00    0.38
  2.10   -1.63
```

Dati interessanti da cui: ztore

- `./app.exe > output.txt` non risolve il problema

=> serve una funzione per scrivere sul file anziché lo schermo

f: rappresenta file

1 fopen
apre il file in scrittura ("w") e dà un «segnalibro» da usare dopo

2 fprintf
scrive nel file, tante volte quante servono

3 fclose
chiude il file: i dati sono al sicuro sul disco

fopen: aprire/creare un file sul disco

fprintf: come printf ma scrive sul file

fclose: chiude il file sul disco

i tre passi

```
FILE *fp;
fp = fopen("grave.txt", "w");
fprintf(fp, "%.2f %.2f\n", t, y);
fclose(fp);
```

no

file aperto in scrittura
w: write

Fibonacci con scrittura sul file

fibonacci_file.c

```
1  #include <stdio.h>
2  #define NMAX 30
3
4  int main() {
5      FILE *fp;
6      int n;
7      unsigned long long a0 = 0, a1 = 1, an;   // F(0) e F(1)
8      fp = fopen("fibonacci.txt", "w");
9      for (n = 1; n <= NMAX; n++) {
10         an = a0 + a1;                        // F(n+1)
11         fprintf(fp, "%d %llu %.6f\n", n, a1, (double) an / a1);
12         a0 = a1;
13         a1 = an;
14     }
15     fclose(fp);
16 }
```

// dichiare fp come file richiede *

// apri file fibonacci.txt sul disco

// fuori dal ciclo.
se dentro ciclo errore per n = 2

```
Terminale: fibonacci_file.c

$ gcc -Wall -Wextra -o app.exe fibonacci_file.c -lm
$ ./app.exe
$ head -4 fibonacci.txt
1 1 1.000000
2 1 2.000000
3 2 1.500000
4 3 1.666667
$ tail -2 fibonacci.txt
29 514229 1.618034
30 832040 1.618034
```

head -n file
prime n righe

opzione "w" : se file esiste viene sovrascritto

"w+" : se file esiste aggiunge nuove righe in fondo (non sovrascrive)

fibonacci_errore.c

```c
#include <stdio.h>
#define NMAX 30

int main() {
    FILE *fp;
    int n;
    unsigned long long a0 = 0, a1 = 1, an;   // F(0) e F(1)
    fp = fopen("dati/fibonacci.txt", "w");   // la cartella dati non esiste
    for (n = 1; n <= NMAX; n++) {
        an = a0 + a1;                        // F(n+1)
        fprintf(fp, "%d %llu %.6f\n", n, a1, (double) an / a1);
        a0 = a1;
        a1 = an;
    }
    fclose(fp);
}
```

mkdir ESER3
cd ESER3
emacs fibonacci.c
gcc -o app.exe -Wall fibonacci.c
./app.exe

cwd: /home/studenti/1csro99/ESER3

/dati

```
Terminale: fibonacci_errore.c

$ gcc -Wall -Wextra -o app.exe fibonacci_errore.c -lm
$ ./app.exe
Segmentation fault (core dumped)
```

cerca di aprire dati/fibonacci.txt
ma dati non esiste!

Soluzione:
mkdir dati
./app.exe

fopen("/fibonacci.txt", "w");
non accessibile all'utente

Controllo file di output

fibonacci_controllo.c

```c
#include <stdio.h>
#include <stdlib.h>
#define NMAX 30

int main() {
    FILE *fp;
    int n;
    unsigned long long a0 = 0, a1 = 1, an;   // F(0) e F(1)
    fp = fopen("dati/fibonacci.txt", "w");   // la cartella dati non esiste
    if (fp == NULL) {
        printf("errore: impossibile aprire il file\n");
        exit(-1);                            // exit termina il programma
    }
    for (n = 1; n <= NMAX; n++) {
        an = a0 + a1;                        // F(n+1)
        fprintf(fp, "%d %llu %.6f\n", n, a1, (double) an / a1);
        a0 = a1;
        a1 = an;
    }
    fclose(fp);
}
```

si esegue solo se file aperto correttamente

controllo dopo fopen e prima di fprintf

NULL: file non aperto correttamente

grave_file.c

```c
#include <stdio.h>
#include <stdlib.h>
#define GRAV 9.81

int main() {
    FILE *fp;
    double h, dt, t, y;
    printf("altezza h (m): ");
    scanf("%lf", &h);
    printf("passo dt (s): ");
    scanf("%lf", &dt);
    fp = fopen("grave.txt", "w");
    if (fp == NULL) {
        printf("errore: impossibile aprire grave.txt\n");
        exit(-1);
    }
    t = 0.;
    y = h;
    while (y > 0.) {
        y = h - 0.5 * GRAV * t * t;
        fprintf(fp, "%.2f %.2f\n", t, y);
        t += dt;
    }
    fclose(fp);
}
```

Terminale: grave_file.c

```
$ gcc -Wall -Wextra -o app.exe grave_file.c -lm
$ ./app.exe
altezza h (m): 20
passo dt (s): 0.1
$ head -3 grave.txt
0.00 20.00
0.10 19.95
0.20 19.80
$ tail -2 grave.txt
2.00 0.38
2.10 -1.63
$ wc -l grave.txt
      22 grave.txt
```

wc = word count

wc -l : conta # righe

wc -l grave.c

wc grave.c :

guardare comando sed

man sed